

Entrega PI – Aplicação de Limites e Derivadas no Jogo

*Autores: André Makoto Molitor, Caio Fábio Freitas,
Davi George Varella da Silva e Davi Moraes Muniz*

Resumo do Jogo

Urban Chase é um jogo de perseguição policial em que o jogador controla um policial utilizando as teclas WASD para movimentação. Durante a perseguição, o jogador pode utilizar mecânicas point and click para posicionar obstáculos e itens estratégicos no cenário, com o objetivo de impedir a fuga do criminoso. O jogo mistura ação, estratégia e raciocínio matemático para criar perseguições dinâmicas.

PARTE 1 – Aplicação de Limites

Limites descrevem o comportamento de uma função quando uma variável se aproxima de um valor específico. No Urban Chase, limites são usados para modelar situações em que algo se aproxima de um estado crítico sem necessariamente atingi-lo — como um fugitivo prestes a escapar, ou a velocidade máxima antes de uma colisão.

1. Limite e Distância de Captura (FugitivoScript.cs)

Contexto: O método VerificarCapturaPorDistancia() no FugitivoScript verifica a cada frame se o policial está próximo o suficiente do fugitivo. Matematicamente, isso representa um limite: quando a distância tende a distanciaParaSerPego, o jogo aciona a captura.

```
// Trecho real do FugitivoScript.cs
float distancia = Vector3.Distance(transform.position,
policial.position);
if (distancia <= distanciaParaSerPego)
    CapturarFugitivo();
```

Modelagem matemática:

Seja $d(t)$ a distância entre o fugitivo e o policial no instante t , e $D = \text{distanciaParaSerPego} = 3$ unidades.

A captura ocorre quando:

$$\lim_{t \rightarrow t_c} d(t) = D$$

Ou seja:

$$\lim_{t \rightarrow t_c} d(t) = 3$$

$$t \rightarrow t_c$$

Resposta: O limite representa o instante crítico t_c em que a distância entre os personagens se aproxima de 3 unidades. Quando $d(t) \rightarrow 3$, o sistema detecta a captura. O limite garante que a verificação seja contínua — o jogo checa frame a frame se o valor está se aproximando do limiar.

2. Limite e Velocidade Mínima de Impacto (ColisaoSom.cs)

Contexto: O ColisaoSom.cs só toca o som de batida e aplica penalidade de pontos se a velocidade de impacto ultrapassar um limiar mínimo.

```
public float velocidadeMinimaImpacto = 2.3f;

private bool PodeTocar(GameObject outro, float velocidade){
    if (velocidade < velocidadeMinimaImpacto) return false;
}
```

Modelagem matemática:

Seja v a velocidade no momento do impacto e $V = 2,3$ u/s o limiar.

$$R(v) = 0, \text{ se } v < V$$

$$R(v) = 1, \text{ se } v \geq V$$

Como os limites laterais são diferentes, o limite não existe em $v = V$.

Resposta: Essa descontinuidade é intencional no design do jogo, criando uma fronteira nítida entre colisões irrelevantes e colisões que causam consequências ao jogador.

3. Limite e Ponto de Fuga (FugitivoScript.cs)

Contexto: O fugitivo navega em direção ao ponto de fuga usando NavMeshAgent. O método VerificarFuga() detecta se ele chegou suficientemente perto do ponto de saída.

```
float distanciaAoPonto = Vector3.Distance(transform.position,
pontoFuga.position);

if (distanciaAoPonto <= distanciaParaEscapar)
    escapou = true;
```

Modelagem matemática:

Seja $p(t)$ a posição do fugitivo e F a posição do ponto de fuga.

$$\lim_{t \rightarrow t_f} p(t) - F = 0$$

Resposta: O limite da distância tende a zero quando o fugitivo chega ao ponto de saída. O sistema usa essa aproximação contínua para registrar a fuga.

PARTE 2 – Aplicação de Derivadas

A derivada representa a taxa de variação instantânea de uma função. No Urban Chase, derivadas aparecem naturalmente sempre que o jogo precisa calcular velocidade, aceleração ou a taxa de mudança de qualquer grandeza ao longo do tempo.

1. Derivada e Velocidade do Fugitivo (FugitivoScript.cs)

```
float velocidadeAtual = fugitivo != null ? fugitivo.velocity.magnitude
: 0f;

somBatida.TentarTocarPorContato(collision.gameObject, velocidadeAtual);
```

Modelagem matemática:

$$s(t) = t^2 + 4t + 1$$

$$v(t) = s'(t)$$

$$v(t) = 2t + 4$$

Resposta: A velocidade instantânea do fugitivo é dada por $v(t) = 2t + 4$.

2. Derivada e Aceleração do Veículo (FugitivoScript.cs)

```
float forca = (fugitivo.velocity.magnitude > 1f) ? 15f : 6f;

hitRb.AddForce((dir + Vector3.up * 0.4f) * forca, ForceMode.Impulse);
```

Modelagem matemática:

$$v(t) = 3t^2 + 2t$$

$$a(t) = v'(t)$$

$$a(t) = 6t + 2$$

Resposta: A aceleração do veículo é $a(t) = 6t + 2$.

3. Derivada e Dificuldade do Jogo (GameplayPartidaController.cs)

```
[SerializeField] private int pontosIniciais = 100;

public void DescontarPontos(int valor) {
    pontosAtuais = Mathf.Max(pontosMinimos, pontosAtuais -
    Mathf.Abs(valor));
    AtualizarTextoPontos();
}
```

Modelagem matemática:

$$d(t) = t^2 + 5$$

$$d'(t) = 2t$$

Resposta: A taxa de crescimento da dificuldade é $d'(t) = 2t$.

4. Derivada pela Definição – Cooldown de Penalidade (FugitivoScript.cs)

```
public float cooldownPenalidade = 0.25f;

if (Time.time - ultimoImpactoTempo < cooldownPenalidade)
    return false;
```

Modelagem matemática:

$$f(x) = 3x^2 + 2x - 1$$

$$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

$$f'(x) = 6x + 2$$

$$f'(3) = 20$$

Resposta: Em $t = 3$ segundos de jogo, a taxa de perda de pontuação por colisão é 20 pontos por unidade de tempo.

Conclusão

O Urban Chase utiliza conceitos de Limites e Derivadas de forma direta e verificável nos scripts do jogo. Os limites modelam os instantes críticos de captura, fuga e detecção de impacto. Já as derivadas descrevem velocidade, aceleração e taxas de variação de pontuação, conectando diretamente a matemática ao comportamento do jogo. Esses conceitos tornam a perseguição mais estratégica e demonstram como o Cálculo I está presente em sistemas reais de desenvolvimento de jogos.